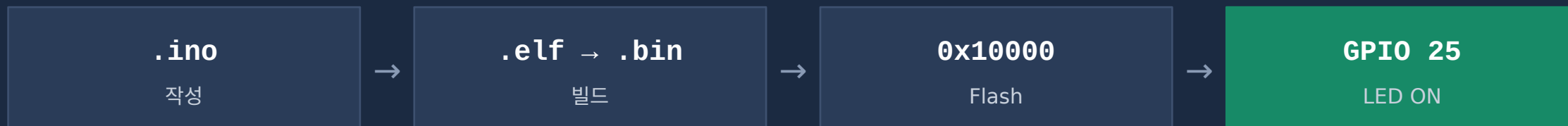


ESP32 빌드 과정과 디지털 I/O

C 과정의 enum · 비트 연산 · 구조체 · 함수 포인터를 오늘 전부 실제 핀에 연결합니다



오늘의 8 교시

오전은 빌드와 출력, 오후는 입력 · 인터럽트 · PWM · 오늘 함정은 대부분 " 아무 일도 일어나지 않음 " 입니다

1

MCU · ESP32 이해

칩 구조, 핀 분류, 3.3V 규칙

2

설치 · 연결 · 빌드

IDE, Micro USB, .ino → Flash

3

배선 · Digital Output

배선도, 220Ω, enum 신호등

4

레지스터 직접 제어

W1TS / W1TC 비트

5

Digital Input

풀업, 논블로킹 디바운스

6

Interrupt

IRAM_ATTR, volatile, 임계 구역

7

PWM (LEDC)

밝기 · RGB · 멜로디

8

미니 프로젝트

함수 포인터 반응속도 게임

MCU(마이크로컨트롤러) 란 ?

컴퓨터 한 대에 필요한 것을 칩 하나에 넣은 것 — OS 없이 전원이 들어오면 바로 우리 프로그램이 돈다

PC — 부품이 따로

CPU

RAM

SSD

GPU

USB · 네트워크

전원 수백 W

OS(Windows · Linux) 위에서 프로그램 실행
printf → 화면 · 파일 → 디스크

MCU — 한 칩 안에

CPU

SRAM

Flash

GPIO

ADC · PWM

UART · I2C

OS 없음 — 전원 ON → 내 프로그램
printf → UART 선 · 결과 → 핀 전압 (LED, 모터)

	PC	MCU (ESP32)
전력	수십 ~ 수백 W	수십 ~ 수백 mW, 잠자기 μ A
부팅	수십 초	1 초 미만
가격	수십만 원 이상	수천 원

MCU 는 어디에 있나 — 주변 전자제품 대부분

1 교시

센서로 읽고 → 판단하고 → 핀으로 움직인다. 오늘부터 4 일 동안 이 세 단계를 직접 만듭니다



가전

세탁기 · 전자레인지 · 에어컨
리모컨



자동차

ECU · 창문 · 시트 · 센서 수십 개



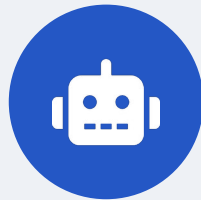
IoT 센서

온습도 · 가스 · 스마트 플러그



웨어러블

스마트밴드 · 무선 이어폰



로봇 · 드론

모터 제어 · 자세 센서



조명 · 스마트홈

Wi-Fi 전구 · 도어락

입력 버튼 · 센서 · ADC Day 1 · 2



판단 C 코드 · 상태 머신 Day 1 ~ 4



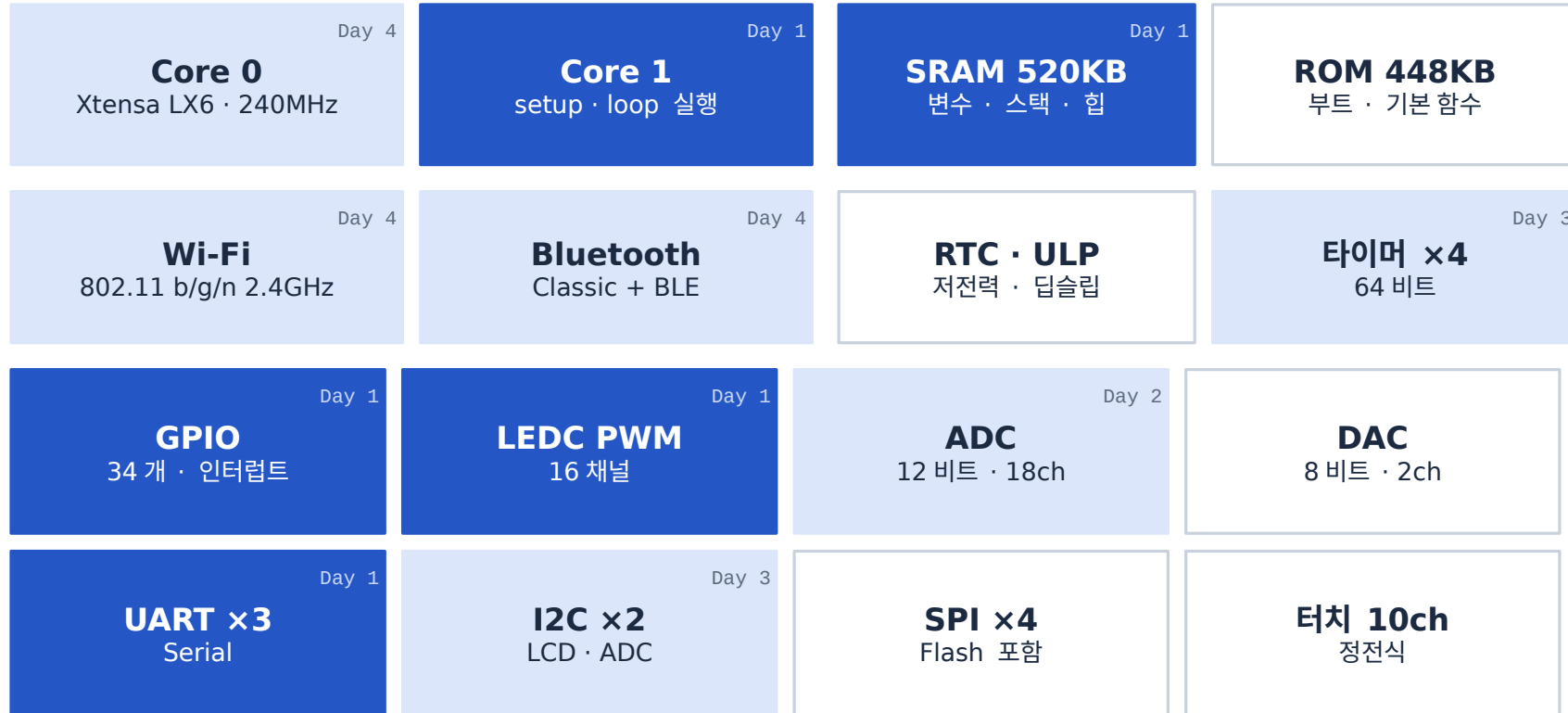
출력 LED · 부저 · 모터 · Wi-Fi Day 1 · 3 · 4

ESP32 칩 안에는 무엇이 있나

1 교시

Espressif ESP32 (ESP32-D0WD 계열) — 오늘 쓰는 블록은 진하게, Day 2~4 에서 쓰는 블록은 열게

ESP32 칩



외부 SPI Flash 4MB
프로그램 (.bin) 저장 · GPIO 6~11 사용

안테나
ESP-32U → 외부 안테나 (U.FL)

오늘 (Day 1) 은 Core 1 ·
SRAM · GPIO · PWM · UART
만 씁니다 . 나머지는 하루에 몇
개씩 씁니다 .

🎬 D1-1 · 칩 블록도 탭 — 블록을 눌러 역할 확인

숫자로 보는 ESP32

1 교시

Arduino Uno 보다 CPU 15 배 · 메모리 250 배 — 게다가 무선이 들어 있다

240

MHz × 2 코어

32 비트 Xtensa LX6

520

KB SRAM

Uno 는 2KB

4

MB Flash

프로그램 저장 (외부 칩)

3.3

V 로직

5V 를 직접 넣지 않는다

34

개 GPIO

입력 전용 · 금지 핀 포함

Wi-Fi

+ Bluetooth

2.4GHz · BLE

핀당 전류는 20mA 이하로 사용 (LED 에는 항상 저항). 5V 출력 센서는 Day 2 에서 분압해서 연결합니다 .

다른 보드와 비교 — ESP32 는 어디쯤인가

MCU(OS 없이 실시간 제어) 와 SBC(Linux 컴퓨터) 는 쓰임새가 다릅니다

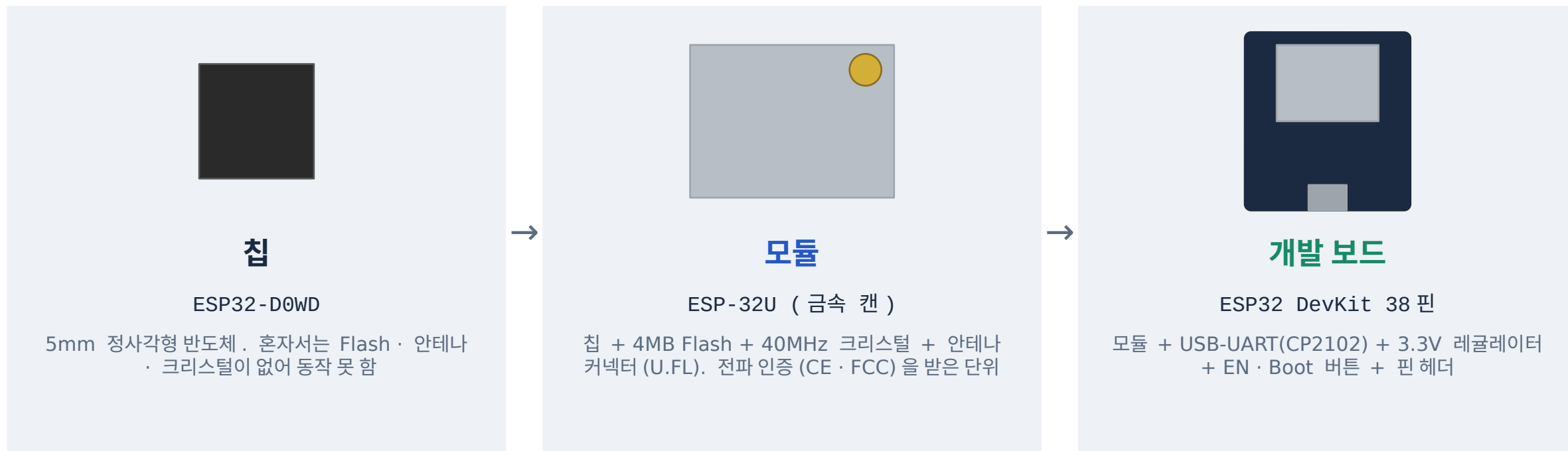
	Arduino Uno	ESP32 (오늘)	Raspberry Pi Pico	Raspberry Pi 4
분류	MCU	MCU	MCU	SBC (Linux 컴퓨터)
CPU	AVR 8 비트 16MHz	Xtensa 32 비트 240MHz ×2	Cortex-M0+ 133MHz ×2	Cortex-A72 1.5GHz ×4
RAM	2KB	520KB	264KB	1~8GB
저장	32KB Flash	4MB Flash	2MB Flash	microSD
로직 전압	5V	3.3V	3.3V	3.3V
무선	없음	Wi-Fi + BT	없음 (Pico W 는 Wi-Fi)	Wi-Fi + BT
OS	없음	없음 (FreeRTOS 내장)	없음	Linux
부팅	즉시	1 초 미만	즉시	수십 초

오늘 키트는 Raspberry Pi 용이지만 부품 (LED · 센서 · 모터) 은 그대로 ESP32 에 씁니다 . 차이는 3.3V 와 핀 번호뿐 .

칩 → 모듈 → 보드

1 교시

우리가 손에 든 것은 " 칩 " 이 아니라 칩을 쓰기 쉽게 감싼 " 개발 보드 " 입니다

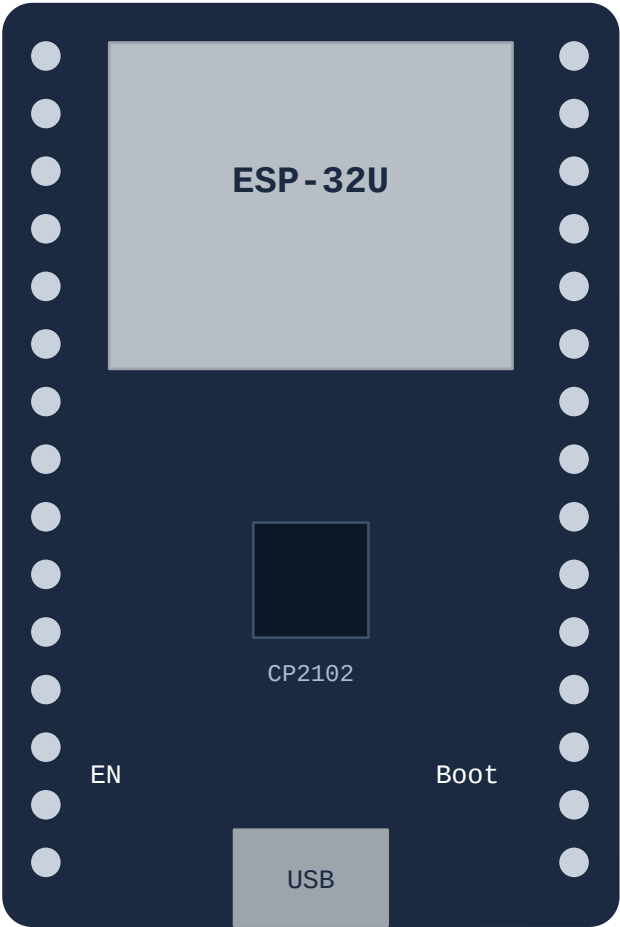


ESP-32U 의 "U" = 기판 안테나가 없고 외부 안테나 커넥터 (U.FL) 가 있는 모델 . Day 4 Wi-Fi 실습 때는 안테나를 꽂아야 신호가 안정적입니다 .

CP2102 가 있어서 USB 케이블 하나로 전원 · 업로드 · 시리얼 모니터가 모두 됩니다 (2 교시).

보드 위에 무엇이 있나

PC 의 C 프로그램은 OS 위에서 , 오늘의 프로그램은 OS 없이 이 칩 하나에서 돕니다



부품	역할
ESP-32U 모듈	Xtensa LX6 듀얼코어 240MHz, SRAM 520KB, Flash 4MB, Wi-Fi · BT
CP2102	USB ↔ UART 변환 . PC 의 COMx 포트가 바로 이 칩
레귤레이터	USB 5V → 3.3V
EN 버튼	리셋 — 칩을 다시 시작
Boot 버튼	누른 채 리셋하면 다운로드 모드 (GPIO 0 = LOW)

printf 는 화면이 아니라 UART → CP2102 → USB → PC 로 나갑니다 . 그래서 Serial.begin(115200) 과 시리얼 모니터 속도가 같아야 합니다 .

핀이 있다고 다 쓸 수 있는 것은 아니다

오늘부터 매일 보는 표 — 출력은 일반 GPIO 에서만 고릅니다

분류	핀	규칙
일반 GPIO	4 13 14 16 17 18 19 21 22 23 25 26 27 32 33	오늘 쓰는 핀은 전부 여기서
입력 전용	34 35 36(VP) 39(VN)	출력 불가 , 내부 풀업 없음 → Day 2 아날로그
Strapping	0 2 5 12 15	부팅 모드 결정 . 12 는 특히 피한다
Flash	6 7 8 9 10 11 (CLK D0~D3 CMD)	절대 금지 — 부팅 불가
UART0	1(TX) 3(RX)	업로드 · 시리얼 모니터가 사용 중
전원	3V3 5V GND EN	5V 는 USB 전원 그대로 . GPIO 에 연결 금지

△ GPIO 34 에 LED → 아무 일도 없음 . GPIO 9 에 부품 → 무한 리셋 . GPIO 12 에 풀업 부품 → 업로드 · 부팅 실패 .

 D1-1 핀맵 · 칩 구조 — " 피해야 할 핀만 보기 "

오늘의 안전 규칙과 키트 사용법

1 교시

배선 실수의 대부분은 이 표로 막을 수 있습니다

전압 안전 3 원칙

1. ESP32 핀은 3.3V, 5V 를 GPIO 에 직접 넣지 않는다
2. 핀 하나당 20mA 이하 . LED 에는 항상 220Ω
3. 배선을 바꿀 때는 USB 를 먼저 뽑는다

키트 부품	ESP32 에서
GPIO Extension Board · 40 핀 케이블	쓰지 않음 (RPI 전용)
830 홀 브레드보드	ESP32 는 옆에 두고 M-F 점퍼선 연결
M-F Dupont Wire ×20	ESP32 핀 ↔ 브레드보드 (8 개 사용)
Jumper Wire (M-M)	j 행 ↔ - 레일 (7 개 사용)
LED · 저항 · 버튼 · 부저 · 센서	그대로 사용

배선도는 3 교시에 " 한 번만 꽃는 " 그림 1 로 시작합니다 . 이후 교시는 그림 1 에서 쓰는 부분만 진하게 표시합니다 .

따라하기 A · 개발 환경 설치

2 교시

다운로드가 크므로 교육 전달 사전 설치 권장 — 수업에서는 USB 연결 (A-5) 부터 확인

A-1	Arduino IDE 2.x 설치	arduino.cc/en/software → Windows 64bit · macOS · Linux Appliance
A-2	한국어 · 스케치북 위치	파일 → 기본 설정 : 한국어 , 경로에 한글이 있으면 C:\esp32 로 , " 자세한 출력 " 컴파일 · 업로드 체크
A-3	보드 매니저 URL	https://espressif.github.io/arduino-esp32/package_esp32_index.json
A-4	esp32 코어 설치	보드 매니저 → esp32 by Espressif Systems → 강사 지정 3.x 버전 (Arduino ESP32 Boards 아님)

증상	조치
esp32 by Espressif 가 검색되지 않음	A-3 URL 오타 확인 → IDE 재시작
다운로드 timeout · network error	다시 설치 → 계속 실패하면 강사 오프라인 패키지 (Arduino15 폴더 복사)
2.x 가 설치됨	ledcAttach 컴파일 에러 → 제거 후 3.x 재설치

A-5 · 보드를 Micro USB 로 연결하기

2 교시

Micro-B 는 위아래 방향이 정해져 있습니다 — 억지로 꽂으면 단자가 떨어져 나갑니다



1. 넓은 쪽끼리 , 좁은 쪽끼리 맞춘다 (대부분 넓은 쪽이 위)
2. 한 손으로 보드를 잡고 , 플러그를 수평으로 곧게
3. 안 들어가면 힘주지 말고 뒤집어서 다시
4. PC 본체 포트에 직접 (전원 없는 허브 · 키보드 포트 피하기)
5. 뽑을 때는 줄이 아니라 플러그 몸통을 잡고 수평으로

보이는 것	조치
빨간 전원 LED 켜짐	정상 → A-6 포트 확인
LED 안 켜짐	끝까지 다시 → 다른 포트 → 다른 케이블
"USB 전원 서지 " 경고 · 보드 뜨거움	즉시 뽑고 3V3 · 5V ↔ GND 합선 확인
만질 때마다 끊김	케이블 교체 , 계속되면 강사에게

연결 전 : 포장 스펀지 제거 · 금속 위에 두지 않기 · 배선 먼저 , USB 는 마지막 . 충전 전용 케이블은 겉모양으로 구분이 안 되므로 " 포트가 생기는 지 " 로 판별 .

A-6 · A-7 · 포트 확인과 보드 설정

2 교시

보드를 꽂으면 포트가 생기고, 뽑으면 사라져야 합니다

OS	확인 방법	정상일 때
Windows	시작 우클릭 → 장치 관리자 → 포트 (COM & LPT)	Silicon Labs CP210x ... (COM5)
macOS	터미널 : <code>ls /dev/cu.*</code>	<code>/dev/cu.usbserial-XXXX</code>
Linux	<code>ls /dev/ttyUSB*</code> · <code>sudo usermod -aG dialout \$USER</code>	<code>/dev/ttyUSB0</code> (재로그인 후)

Windows " 기타 장치 " 에 노란 느낌표 → [silabs.com](https://www.silabs.com/usb-serial/WindowsDriver) 의 CP210x Universal Windows Driver → `silabser.inf` 우클릭 → 설치

도구 메뉴	값
보드	esp32 → ESP32 Dev Module
포트	위에서 확인한 포트
Upload Speed	921600 (뚫기면 115200)
CPU Frequency	240MHz (WiFi/BT)
Flash Size · Partition	4MB · Default 4MB with spiffs

A-8 설치 확인

- 메뉴 한국어, 경로에 한글 없음
- esp32 3.x.x INSTALLED
- Micro USB 방향 맞춤 · 전원 LED ON
- 꽂으면 포트 생김, 뽑으면 사라짐
- 보드 ESP32 Dev Module · 포트 선택

따라하기 B · 새 스케치부터 업로드까지

2 교시

오늘 모든 예제를 이 순서로 실행합니다 — 단축키를 익혀 두면 하루가 편합니다



```
C:\esp32\  
└─ hello_esp32\  
    └─ hello_esp32.ino  
  
# 컴파일 성공  
Sketch uses 290841 bytes (22%) of program storage space.  
  
# 컴파일 오류 — 파일 : 줄 : 칸 : error:  
hello_esp32.ino:4:5: error: 'Serail' was not declared in this  
scope
```

```
esptool.py v4.x  
Serial port COM5  
Connecting.... ← 멈추면 Boot 누르고 있기  
Chip is ESP32-D0WD-V3 (revision v3.1)  
Changing baud rate to 921600  
Writing at 0x00010000... (10 %)  
...  
Writing at 0x0004a1f0... (100 %)  
Hash of data verified.  
Hard resetting via RTS pin... ← 끝
```

.ino 가 칩에 올라가기까지

2 교시

파일 → 기본 설정 → "자세한 출력 보이기 : 컴파일, 업로드" 를 켜고 로그에서 찾습니다



Flash 4MB 앞부분 — esptool write_flash 가 쓰는 주소

0x0000	0x1000 bootloader	0x8000 partitions	0xe000 boot_app0	0x10000 앱 (.ino.bin)
--------	----------------------	----------------------	---------------------	-------------------------

```
esptool --chip esp32 --port COM5 --baud 921600 write_flash \  
0x1000 bootloader.bin 0x8000 partitions.bin \  
0xe000 boot_app0.bin 0x10000 hello_esp32.ino.bin
```

PC 는 OS 가 실행 파일을 메모리에 올려 줍니다 . MCU
에는 OS 가 없으므로 Flash 의 정해진 주소에 직접 써
두고 ROM → bootloader → 앱 순서로 시작합니다 .

🎬 D1-2 설치 · USB 연결 · 빌드 — 빌드 파이프라인 12 단계

setup() 과 loop() 는 누가 부르나

2 교시

main() 은 사라진 것이 아니라 core 안에 숨어 있습니다

```
// Arduino-ESP32 core 내부 ( 간략화 )
extern "C" void app_main() {
    initArduino();
    xTaskCreatePinnedToCore(loopTask, "loopTask",
                           8192, NULL, 1, NULL, 1);
}

void loopTask(void *arg) {
    setup();           // 한 번
    for (;;) loop();  // 무한 반복
}
```

setup/loop 는 FreeRTOS 태스크 하나 안에서 돌아가는 평범한 함수입니다 . Day 4 에서 이 태스크 옆에 우리 태스크를 더 만듭니다 .

```
void setup() {
    Serial.begin(115200);
    delay(500);
    Serial.printf("setup: task=%s core=%d\n",
                  pcTaskGetName(NULL), xPortGetCoreID());
}
// → setup: task=loopTask core=1
```

증상	조치
포트가 안 보임	데이터 케이블 · CP2102 드라이버
Failed to connect	Connecting 동안 Boot 버튼 누르기
글자가 깨짐	시리얼 모니터 115200 으로

🎬 D1-2 · app_main → loopTask → setup → loop

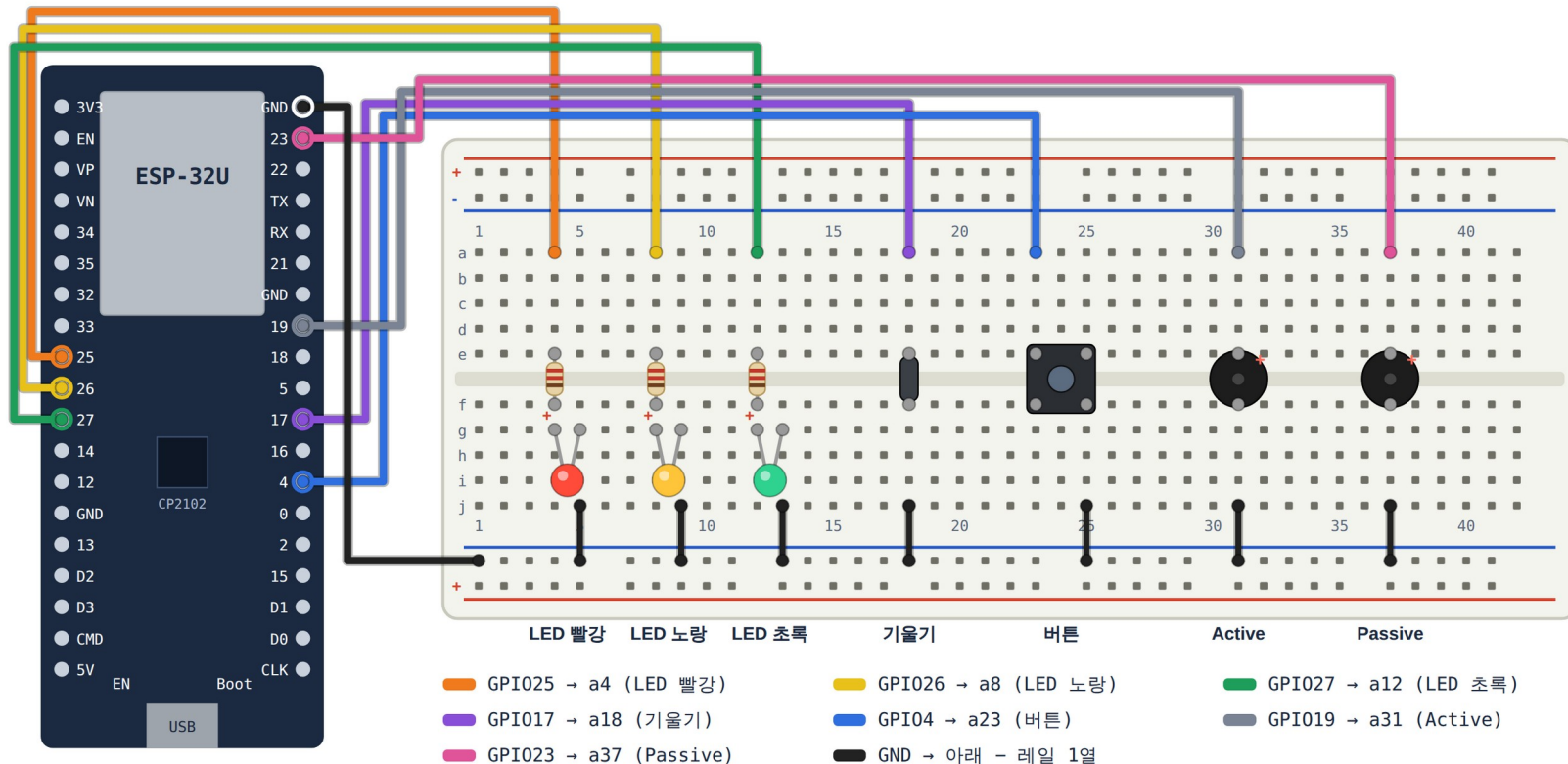
배선도 — 오늘 한 번만 꽃는다

3 교시

모든 부품이 같은 규칙 : GPIO 선은 a 행 → 부품이 가운데 홈을 건넌 → j 행에서 파란 - 레일 (GND)

그림 1 · Day 1 기본 배선 (3·4·5·6·7·8교시 공통)

모든 부품이 같은 규칙: GPIO 선은 a행 → 부품이 가운데 홈을 건넌 → j행에서 파란 - 레일(GND)



ESP32 는 38 핀이라 브레드보드에 꽃으면 한쪽 열만 남으므로 , 보드는 옆에 두고 M-F 점퍼선으로 연결합니다 (구멍 쪽 → ESP32 핀 , 핀 쪽 → 브레드보드).

따라 꽃기 — 순서대로

3 교시

USB 를 뽑은 상태에서 시작 · 브레드보드는 숫자 1 이 왼쪽 , 파란 - 레일이 아래

순서	부품	위치
1	저항 $220\Omega \times 3$ (빨강 빨강 갈색)	e4-f4 · e8-f8 · e12-f12
2	LED $\times 3$ (긴 다리 + 가 왼쪽)	g4·g5 · g8·g9 · g12·g13
3	기울기 센서	e18-f18
4	버튼 (딸깍 소리까지)	e23 · e25 · f23 · f25
5	Active 부저 · Passive 부저 (+ 가 위)	e31-f31 · e37-f37
6	짧은 점퍼 M-M $\times 7 \rightarrow -$ 레일	j5 j9 j13 j18 j25 j31 j37

7. USB 꽃기 전 점검

- 어떤 선도 3V3 · 5V 핀에 꽂혀 있지 않다
- LED 긴 다리가 저항 쪽 (왼쪽) · 점퍼는 파란 - 레일
- ESP32 핀 번호를 보드 인쇄 글자로 다시 읽었다

ESP32 핀	보드 위치	→
25	왼쪽 위에서 9 번째	a4
26	왼쪽 10 번째	a8
27	왼쪽 11 번째	a12
17	오른쪽 11 번째	a18
4	오른쪽 13 번째	a23
19	오른쪽 8 번째	a31
23	오른쪽 2 번째	a37
GND	오른쪽 맨 위	- 레일 1 열

레일이 30 열 근처에서 끊긴 브레드보드는 좌우 - 레일을 짧은 선으로 이어야 31 · 37 열 부저가 GND 에 연결됩니다 .

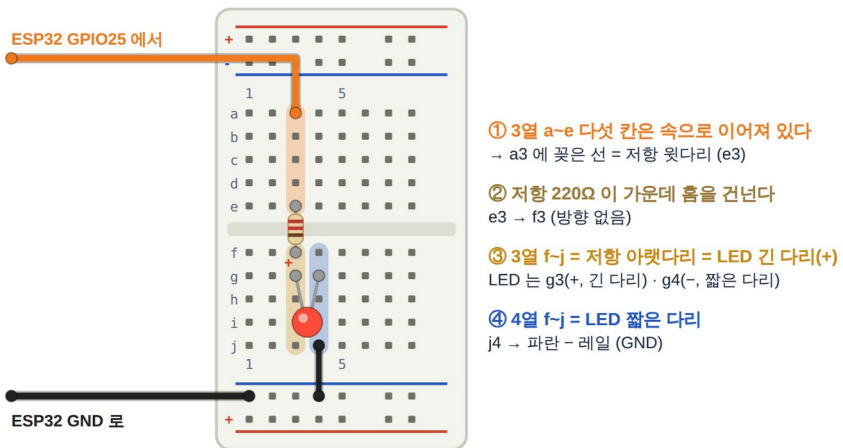
🎬 D1-3 · 브레드보드 따라 꽃기 — 8 단계 , 구멍 누르면 이어진 칸 표시

브레드보드 속은 어떻게 이어져 있나

3 교시

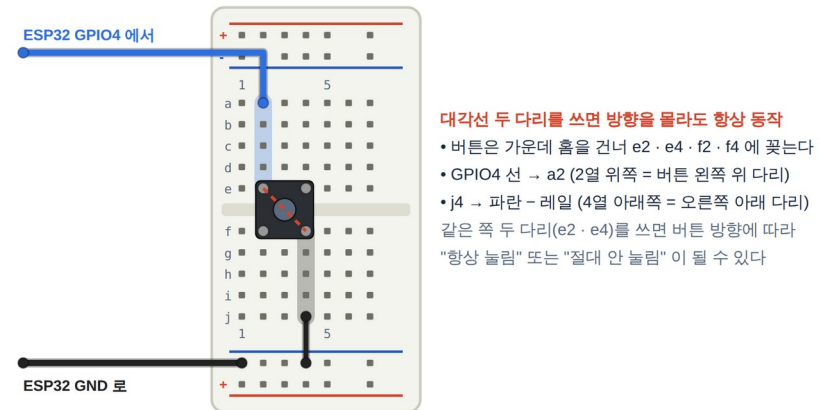
같은 열의 a~e 다섯 칸, f~j 다섯 칸이 속에서 이어져 있고 가운데 홈에서 끊깁니다

그림 2 · LED 한 개 확대 — 브레드보드 속에서 이어진 곳



전류 경로: GPIO25 → a3 - e3 → 저항 → f3 - g3 → LED(+ → -) → g4 - j4 → - 레일 → GND

그림 3 · 버튼 확대 — 대각선 연결



저항이 홈을 건너야 위 (GPIO) 와 아래 (LED) 가 이어진다 . 전류 : GPIO25 → a3 - e3 → 저항 → f3 - g3 → LED → g4 - j4 → - 레일

버튼은 대각선 두 다리를 쓰면 방향과 상관없이 동작 . 같은 쪽 두 다리는 " 항상 눌림 " 이 될 수 있다 .

LED 에는 왜 220Ω 인가

3 교시

LED 전압 (Vf) 은 색마다 거의 일정합니다 . 나머지 전압이 저항에 걸리고 , 그 저항이 전류를 정합니다

$$I = (3.3V - V_f) \div R$$

$$\text{빨강} : (3.3 - 2.0) \div 220 \approx 5.9 \text{ mA}$$

순서	LED 가 안 켜질 때 확인
1	LED 방향 — 긴 다리 (+) 가 저항 쪽
2	브레드보드 — 같은 5 칸 줄 , 가운데 홈 위아래는 끊김
3	LED(-) 가 보드 GND 까지
4	핀 번호 — 입력 전용 34~39 아닌지
5	pinMode(pin, OUTPUT) 빠뜨리지 않았는지

LED	Vf	220Ω	100Ω	10kΩ
빨강	2.0V	5.9mA	13mA	0.13mA
노랑	2.1V	5.5mA	12mA	0.12mA
초록	2.2V	5.0mA	11mA	0.11mA

220Ω 빨강 빨강 갈색 **10kΩ** 갈색 검정 주황 **100Ω** 갈색 검정 갈색

⚠ 저항 없이 LED 를 연결하면 전류를 막을 것이 없어 LED 나 GPIO 가 손상될 수 있습니다 . 오늘은 한 번도 하지 않습니다 .

🎬 D1-3 따라 꽃기 · LED · 신호등 — 저항을 0Ω · 10kΩ 으로 바꿔 보기

신호등은 상태다 — enum + switch

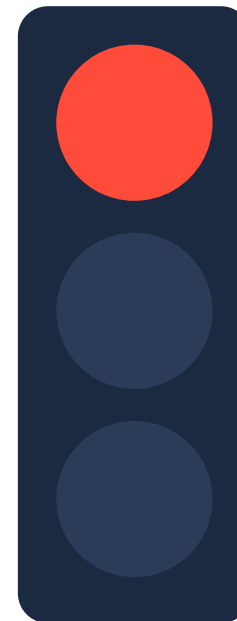
3 교시

상태에 이름을 붙이고, 상태마다 할 일을 case 로 나눕니다

```
typedef enum { ST_RED, ST_GREEN, ST_YELLOW } light_t;

void loop() {
    static light_t state = ST_RED;    // loop 가 끝나도 유지
    switch (state) {
        case ST_RED:
            setLights(1, 0, 0); delay(3000); state = ST_GREEN; break;
        case ST_GREEN:
            setLights(0, 0, 1); delay(3000); state = ST_YELLOW; break;
        case ST_YELLOW:
            setLights(0, 1, 0); delay(1000); state = ST_RED; break;
    }
}
```

8 교시에는 이 switch 를 함수 포인터 표로 바꿉니다.



RED → GREEN → YELLOW → RED
...

⚠ static 을 빼면 loop() 가 불릴 때마다 state 가 ST_RED 로 다시 초기화 → 빨강만 반복 . C 과정의 지역 변수 수명 그대로 .

GPIO 는 메모리 주소에 있는 32 비트다

4 교시

C 과정 Day 3 의 비트 연산을 레지스터에 그대로 씁니다

```
#include "soc/soc.h"
#include "soc/gpio_reg.h"

#define LED_MASK ((1UL << 25) | (1UL << 26) | (1UL << 27))

void setup() {
    pinMode(25, OUTPUT); pinMode(26, OUTPUT); pinMode(27, OUTPUT);
}
void loop() {
    REG_WRITE(GPIO_OUT_W1TS_REG, LED_MASK); // 3 개 동시에 ON
    delay(500);
    REG_WRITE(GPIO_OUT_W1TC_REG, LED_MASK); // 3 개 동시에 OFF
    delay(500);
}
```

레지스터	쓰기 동작
GPIO_OUT_REG	32 비트 전체를 덮어쓰
GPIO_OUT_W1TS_REG	1 인 비트만 켜기 (Set)
GPIO_OUT_W1TC_REG	1 인 비트만 끄기 (Clear)

LED_MASK = 0x0E000000

31	30	29	28	27	26	25	24
0	0	0	0	1	1	1	0

W1TS 에 쓰면 0 인 비트는 영향 없음 → 다른 핀을 읽을 필요 없고, 인터럽트가 끼어들어도 안전 .

🎬 D1-4 W1TS / W1TC 비트 뷰어

⚠ 레지스터를 잘못 쓰면 남의 핀이 꺼진다

4 교시

한 핀만 바꿀 때는 W1TS / W1TC · 속도는 직접 재서 자기 숫자를 적습니다

```
REG_WRITE(GPIO_OUT_REG, 1UL << 25); // 25 번만 켜려는 의도
// → 26, 27 번을 포함해 0~31 번 출력이 전부 0
```

실수	결과
GPIO_OUT_REG 에 한 비트만	다른 핀 전부 LOW
pinMode 없이 레지스터만	출력 비활성 — 안 켜짐
1UL << 32 로 GPIO 32	32 비트 초과 → OUT1 레지스터

대부분은 digitalWrite 로 충분합니다. 여러 핀을 같은 순간에 바꾸거나 빠른 신호를 직접 만들 때 (Day 3 74HC595) 만 레지스터.

```
uint32_t t0 = micros();
for (long i = 0; i < 100000; i++) {
    digitalWrite(25, HIGH); digitalWrite(25, LOW);
}
uint32_t t_api = micros() - t0;

t0 = micros();
for (long i = 0; i < 100000; i++) {
    REG_WRITE(GPIO_OUT_W1TS_REG, 1UL << 25);
    REG_WRITE(GPIO_OUT_W1TC_REG, 1UL << 25);
}
uint32_t t_reg = micros() - t0;
```

측정	내 결과
digitalWrite	_____ μ s
register	_____ μ s
배수	_____ ×

🎬 D1-4 · GPIO_OUT_REG 덮어쓰기 버튼

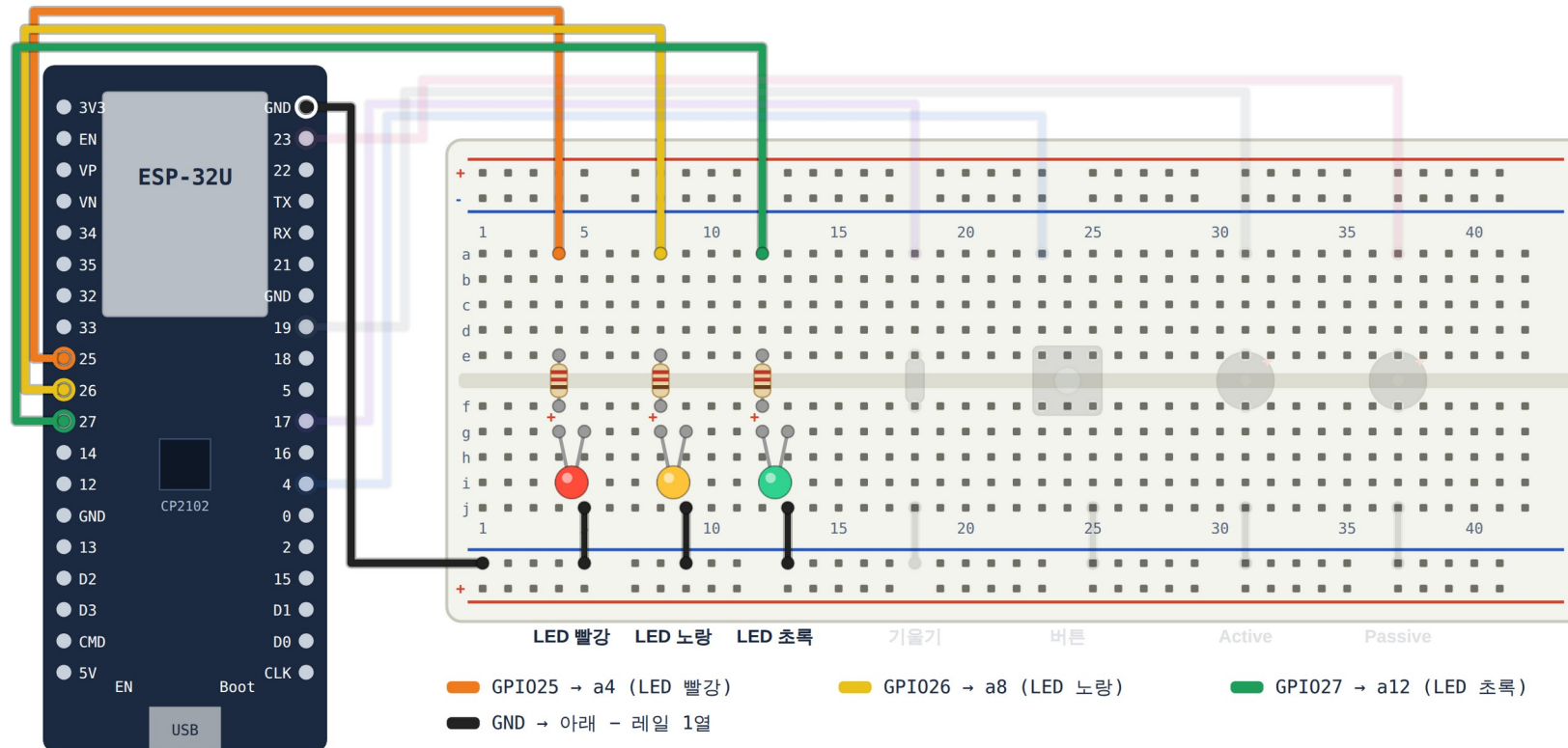
4 교시 배선 — 3 교시 그대로

4 교시

예제 4-2 · 빈칸 4-1 은 LED 3 개 , 예제 4-3 은 LED 25 하나만 씩입니다

예제 4-2 배선 — 레지스터로 LED 3개 (bit 25 · 26 · 27)

진한 부분만 이 예제에서 씁니다. 흐린 부분은 그림 1 대로 꽂혀 있어도 상관없습니다



레지스터 비트 번호 = GPIO 번호 : bit 25 · 26 · 27 이 LED 빨강 · 노랑 · 초록

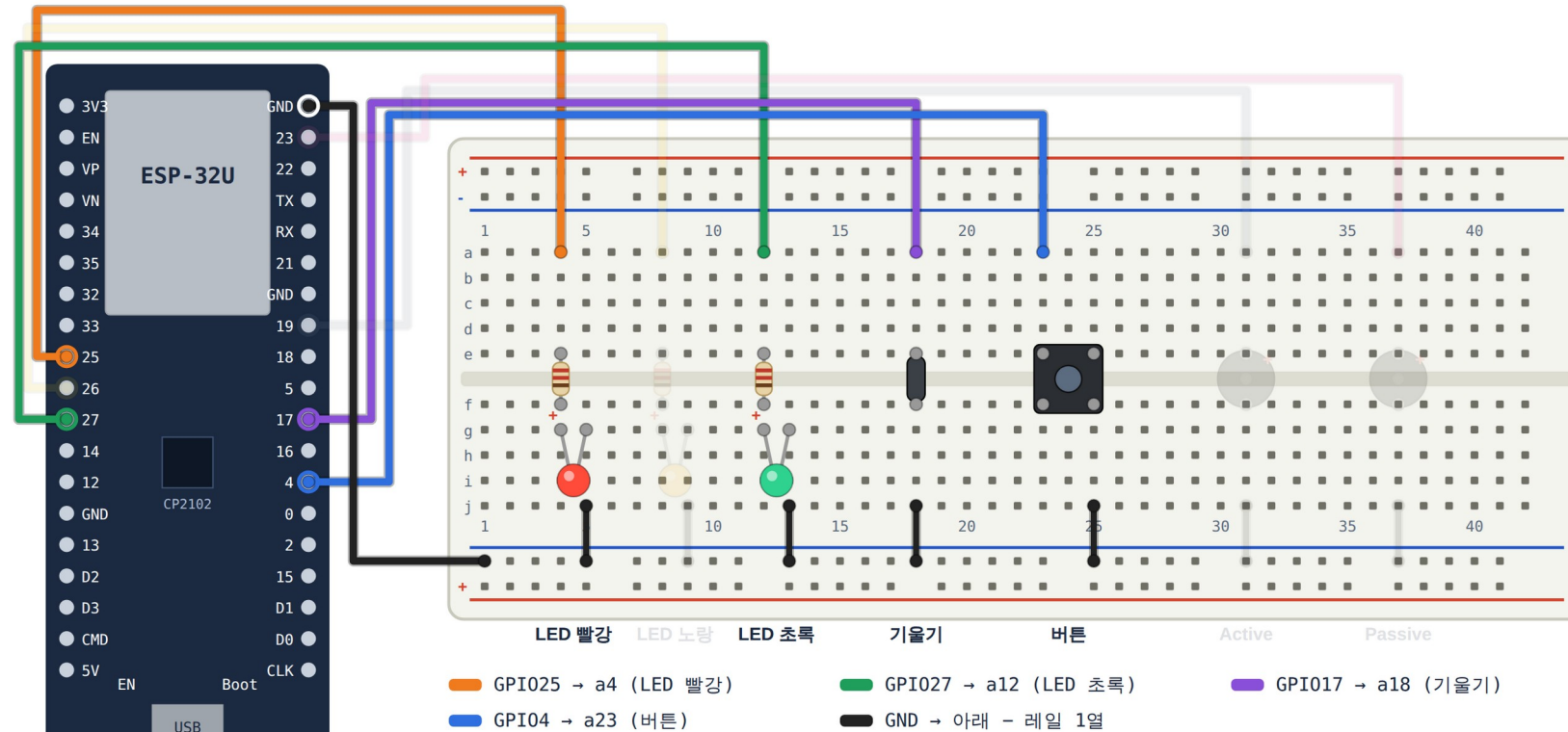
5 교시 배선 — 버튼 · 기울기 센서

5 교시

예제 5-1 · 5-2 는 버튼만 , 5-3 은 버튼 + LED 25, 5-4 는 아래 전부

예제 5-4 배선 — 버튼(4) + 기울기(17) + LED 25 · 27

진한 부분만 이 예제에서 씁니다. 흐린 부분은 그림 1 대로 꽂혀 있어도 상관없습니다



버튼은 a23(GPIO4) 과 j25(GND) 대각선 . 기울기 센서는 방향 없음

입력은 풀업부터 — 그리고 채터링

5 교시

버튼이 떨어져 있으면 핀은 플로팅 . INPUT_PULLUP 이면 안 누를 때 1, 누를 때 0

모드	안 누름	누름	배선
INPUT	불안정	불안정	외부 저항 필요
INPUT_PULLUP	1 (HIGH)	0 (LOW)	핀 ↔ 버튼 ↔ GND
INPUT_PULLDOWN	0	1	핀 ↔ 버튼 ↔ 3V3

```
if (last == HIGH && now == LOW) {    // 하강 엣지
    edges++;
    Serial.printf("press #%d\n", edges);
}
// 한 번 눌렀는데 → press #1 / #2 / #3
```

한 번 누를 때 핀이 실제로 읽는 값 (수 ms)



기계식 접점은 붙는 순간 수 ms 동안 붙었다 떨어지기를 반복합니다 . loop() 는 그보다 훨씬 빨라서 떨림을 여러 번의 누름으로 셉니다 .

⚠ GPIO 34~39 는 내부 풀업이 없어 INPUT_PULLUP 이 무시됩니다 .
오늘은 4, 17 번을 씁니다 .

논블로킹 디바운스 — 상태는 구조체에 , 함수는 하나

5 교시

delay(30) 으로 기다리지 않고 시각을 기록해 비교합니다 . loop() 는 멈추지 않습니다

```
typedef struct {
    int pin;
    int stable;           // 확정된 상태
    int last_raw;         // 직전 원시 값
    uint32_t changed_at;  // 원시 값이 바뀐 시각
} button_t;

int button_pressed(button_t *b) {
    int raw = digitalRead(b->pin);
    if (raw != b->last_raw) {
        b->last_raw = raw;
        b->changed_at = millis();
    }
    if (millis() - b->changed_at > 30 && raw != b->stable) {
        b->stable = raw;
        return (raw == LOW);
    }
    return 0;
}
```

```
button_t btn = {4, HIGH, HIGH, 0};
button_t tilt = {17, HIGH, HIGH, 0};

void loop() {
    if (button_pressed(&btn))
        Serial.println("button");
    if (button_pressed(&tilt))
        Serial.println("tilted");
}
```

millis() - changed_at > 30 처럼 뺄셈으로 비교하면 49.7 일 뒤 millis() 가 0 으로 돌아와도 맞습니다 .

버튼 30ms, 기울기 센서 100ms — 설정값도 구조체 멤버로 (빈칸 5-1).

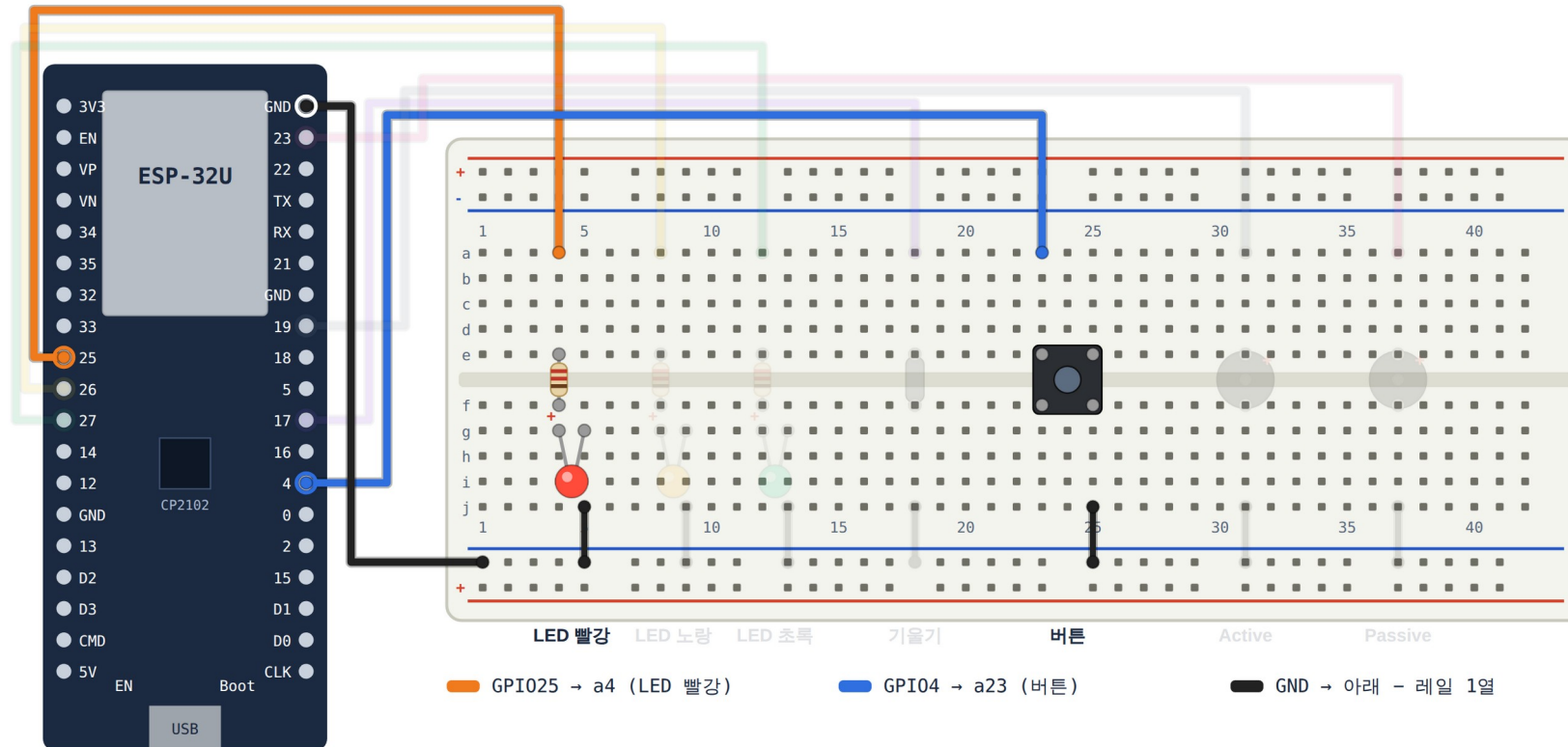
6 교시 배선 — 버튼과 LED

6 교시

예제 6-1 · 6-2 는 버튼만 , 빈칸 6-1 은 버튼 + LED 25

빈칸 6-1 배선 — 인터럽트로 LED 토글 (버튼 4 · LED 25)

진한 부분만 이 예제에서 씁니다. 흐린 부분은 그림 1 대로 꽂혀 있어도 상관없습니다



INPUT_PULLUP 버튼을 누르면 HIGH → LOW — 그래서 인터럽트 모드는 FALLING

loop 가 바빠도 ISR 은 끼어든다

6 교시

폴링은 읽는 순간 눌러 있어야만 셉니다. 인터럽트는 하드웨어가 알려 줍니다

```
volatile uint32_t press_count = 0;
volatile uint32_t last_isr_ms = 0;
portMUX_TYPE mux = portMUX_INITIALIZER_UNLOCKED;

void IRAM_ATTR onButton() {
    uint32_t now = millis();
    if (now - last_isr_ms > 50) {
        portENTER_CRITICAL_ISR(&mux);
        press_count++;
        portEXIT_CRITICAL_ISR(&mux);
        last_isr_ms = now;
    }
}

void setup() {
    pinMode(4, INPUT_PULLUP);
    attachInterrupt(digitalPinToInterrupt(4), onButton, FALLING);
}
```

요소	의미
IRAM_ATTR	ISR 을 내부 RAM 에 — 즉시 실행
FALLING	HIGH → LOW 순간 (폴업 버튼 눌림)
volatile	매번 메모리에서 읽어라
portENTER_CRITICAL	이 구간에 끼어들기 금지
onButton	함수 포인터 — qsort 의 비교 함수처럼

폴링 + delay(2000): 2 초에 5 번 눌러도 0~1 증가.
인터럽트 : 5 번 누르면 pressed 5 time(s).

ISR 은 표시만 하고 나온다 · 경쟁 상태

6 교시

출력 · 대기 · 계산은 loop() 에서 . 읽고 - 바꾸기는 임계 구역으로 묶습니다

```
void IRAM_ATTR onButton() {  
    Serial.println("pressed!"); // x 잠금 대기 → 멈춤 / 크래시  
    delay(100);                // x ISR 안에서는 의미 없음  
    count++;                   // count 가 volatile 이 아니면  
}  
                               // loop 가 못 볼 수 있음
```

임계 구역이 없을 때 (**press_count = 3** 에서 시작)

loop	n = press_count;	n = 3
ISR	press_count++;	press_count = 4
loop	press_count = 0;	4 번째가 지워짐
loop	printf n	출력 3 (실제 4)

```
void loop() {  
    portENTER_CRITICAL(&mux);  
    uint32_t n = press_count; // 읽고  
    press_count = 0;          // 비우기를  
    portEXIT_CRITICAL(&mux);  // 한 덩어리로  
  
    if (n) Serial.printf("pressed %lu time(s)\n", n);  
    delay(2000);  
}
```

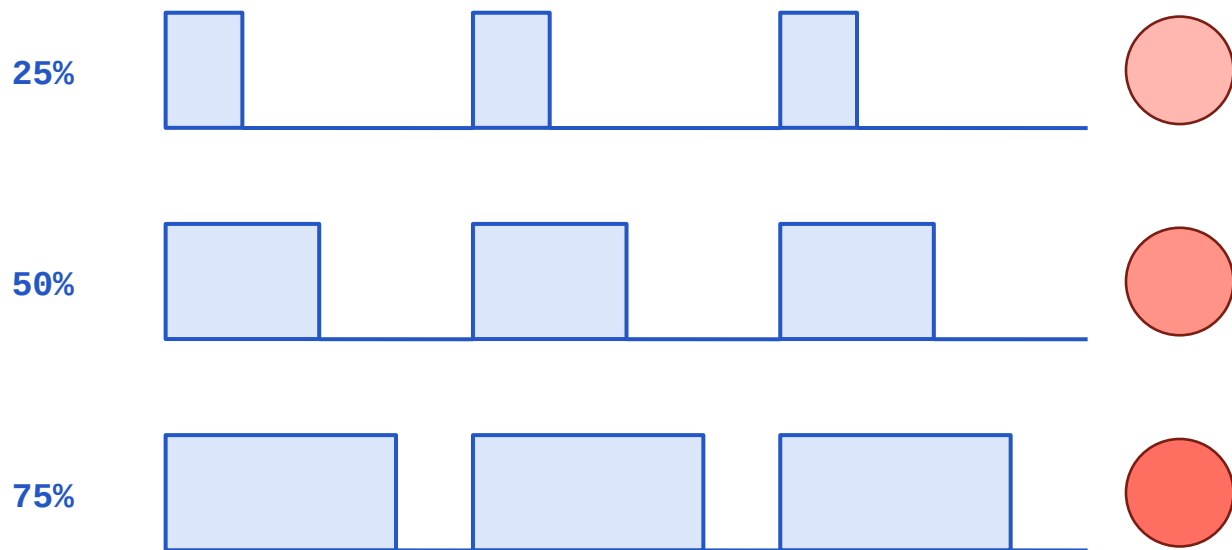
임계 구역 안에서 놀리면 ISR 은 portEXIT 까지 기다렸다 실행 → 놀림을 잃지 않음 . bool 플래그 하나 쓰고 지우기는 임계 구역 없이도 안전 .

🎬 D1-6 · 경쟁 상태 — portENTER_CRITICAL 토글

PWM — 켜진 시간의 비율이 밝기가 된다

7 교시

5kHz 로 깜빡이면 눈은 평균 밝기만 봅니다 . core 3.x 는 핀 중심 API



용어	의미	예
주파수	1 초에 켜고 끄는 횟수	5000 Hz
분해능	듀티를 나눈 단계 (비트)	8 비트 → 0~255
듀티	켜진 시간의 비율	128/255 ≈ 50%

```
ledcAttach(25, 5000, 8); // 핀 , 주파수 , 분해능

for (int d = 0; d <= 255; d++) {
    ledcWrite(25, d); // 핀 , 듀티
    delay(5);
}
```

⚠ 인터넷의 ledcSetup / ledcAttachPin 예제는 core 2.x 방식 →
"'ledcSetup' was not declared in this scope". core 버전을 고정하는 이유 .

📺 D1-7 PWM 듀티비 시각화 — 주파수 1Hz 로 내려 보기

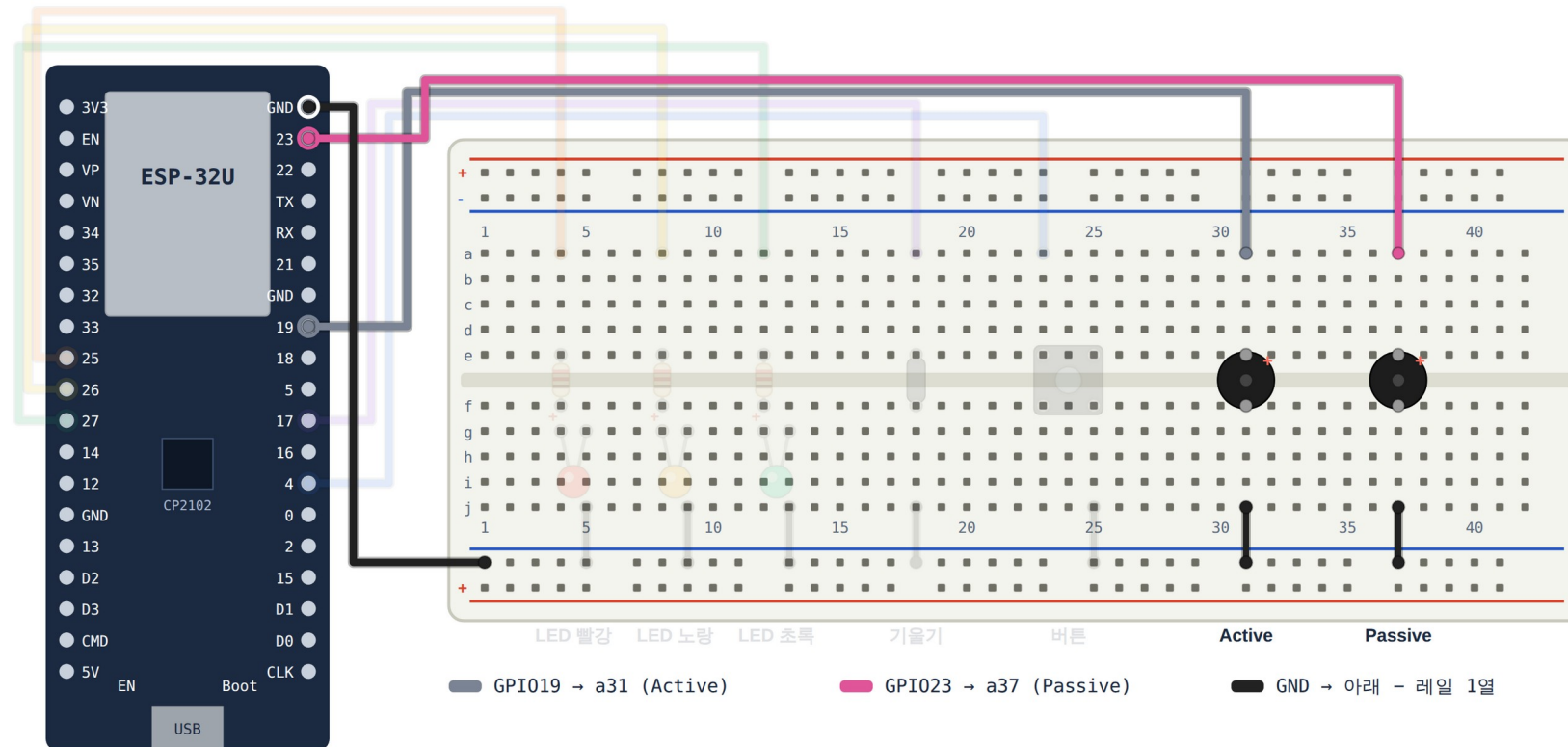
7 교시 배선 — 부저 두 개

7 교시

예제 7-3 은 두 부저 모두 , 7-4 · 빈칸 7-1 은 Passive(23) 만

예제 7-3 배선 — Active 부저(19) · Passive 부저(23)

진한 부분만 이 예제에서 씁니다. 흐린 부분은 그림 1 대로 꽂혀 있어도 상관없습니다



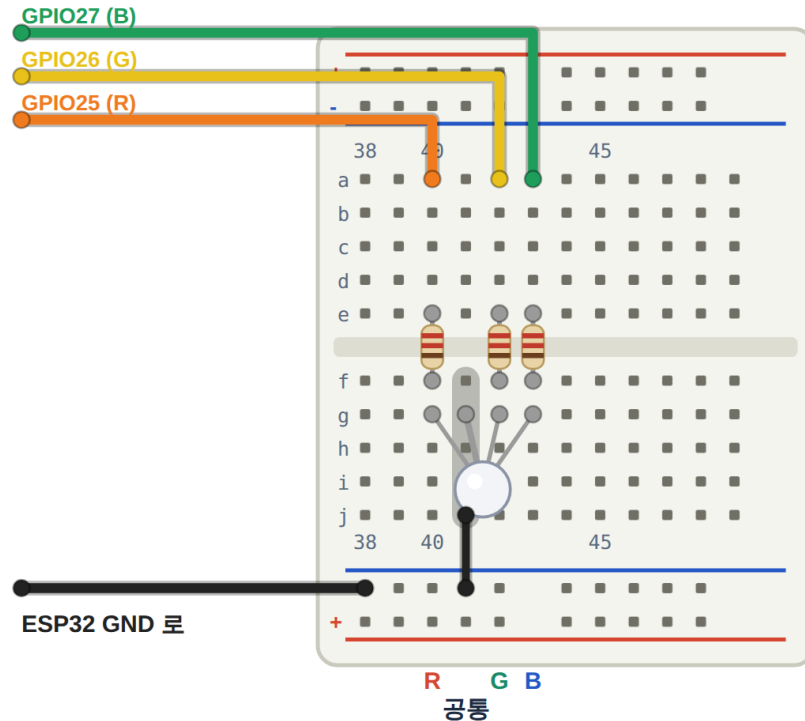
+ 표시가 위 (e 행). Active = 뒷면 막힘 (정해진 음), Passive = 뒷면 기판 보임 (주파수를 우리가 만듦)

7 교시 배선 — RGB LED (예제 7-2)

7 교시

비어 있는 40~43 열에 꽂고 GPIO 25 · 26 · 27 선만 옮깁니다

그림 4 · RGB LED 배선 (7교시 예제 7-2)



가장 긴 다리 = 공통 단자 (41열)

- 다리 순서: R · 공통 · G · B → g40 · g41 · g42 · g43
- R · G · B 다리 열(40 · 42 · 43)에만 220Ω 을 건넌다
- 공통 캐소드형: j41 → 파란 - 레일 (그림)
- 공통 애노드형: j41 → 빨간 + 레일, + 레일은 3V3 에 연결

LED 3개에 꽂혀 있던 25 · 26 · 27 선을 뽑아 a40 · a42 · a43 으로 옮긴다 (다른 부품은 그대로)

가장 긴 다리가 공통 단자 . 캐소드형은 - 레일 , 애노드형은 + 레일 (3V3) 에 연결하고 코드의 COMMON_ANODE 를 1 로

색과 소리도 데이터다 — 구조체 배열

7 교시

악보가 배열 (데이터) 이고 연주가 루프 (코드) 입니다 . 곡을 바꿀 때 코드를 고치지 않습니다

```
typedef struct {
    uint8_t r, g, b;
    const char *name;
} color_t;

const color_t COLORS[] = {
    {255, 0, 0, "red"},
    {255, 180, 0, "orange"},
    {160, 0, 255, "purple"},
};

#define COMMON_ANODE 0 // 애노드형이면 1
void set_rgb(const color_t *c) {
    #if COMMON_ANODE
        /* 255 - 값 으로 반전 */
    #endif
    ledcWrite(25, c->r); /* ... */
}
```

```
typedef struct { uint16_t freq; uint16_t ms; } note_t;

const note_t melody[] = {
    {262, 300}, {294, 300}, {330, 300}, {349, 300},
    {392, 300}, {440, 300}, {494, 300}, {523, 600},
};
const int N = sizeof(melody) / sizeof(melody[0]);

for (int i = 0; i < N; i++) {
    ledcWriteTone(23, melody[i].freq);
    delay(melody[i].ms);
}
ledcWriteTone(23, 0); // 0 Hz = 끄기, 쉼표는 {0, 300}
```

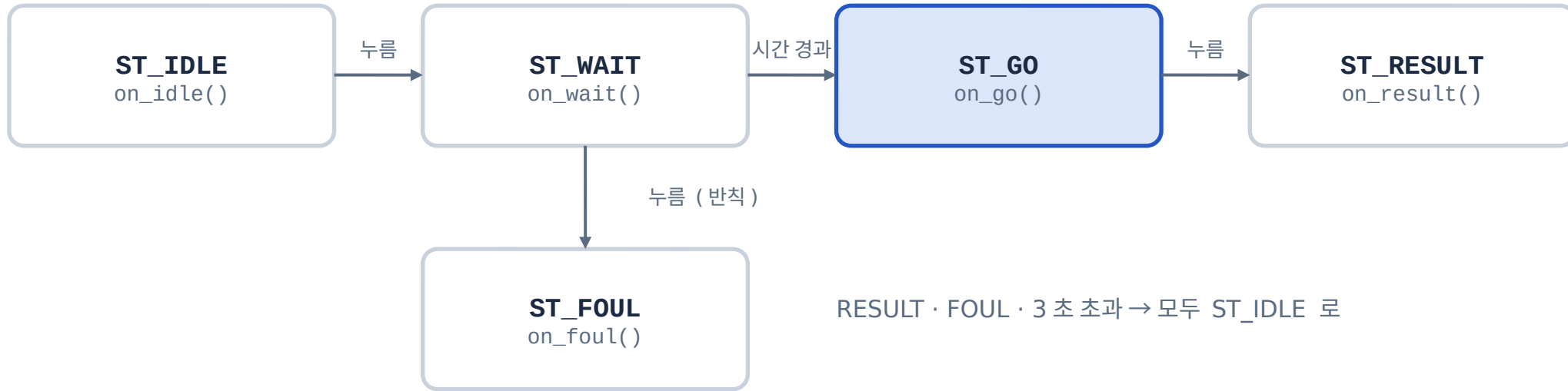
부저	구분	소리 내는 법
Active (19)	뒷면 막힘	digitalWrite HIGH — 정해진 음
Passive (23)	뒷면 기판 보임	ledcWriteTone — 주파수를 만듦

🎬 D1-7 · 건반 — ledcWriteTone(23, freq)

current = table[current]();

8 교시

switch 대신 함수 포인터 표 — enum 값이 그대로 배열 인덱스가 됩니다



```
typedef enum { ST_IDLE, ST_WAIT, ST_GO, ST_RESULT, ST_FOUL, ST_COUNT } state_t;
typedef state_t (*state_fn)(void);

const state_fn table[ST_COUNT] = { on_idle, on_wait, on_go, on_result, on_foul };
state_t current = ST_IDLE;

void loop() { current = table[current](); }
```

상태 추가 = enum 이름 하나 + 함수 하나
+ 표 한 칸 . loop() 는 한 줄 그대로 .
ST_COUNT 는 상태 개수를 자동으로 셉니다 .

🎬 D1-8 함수 포인터 상태 머신 — 직접 플레이

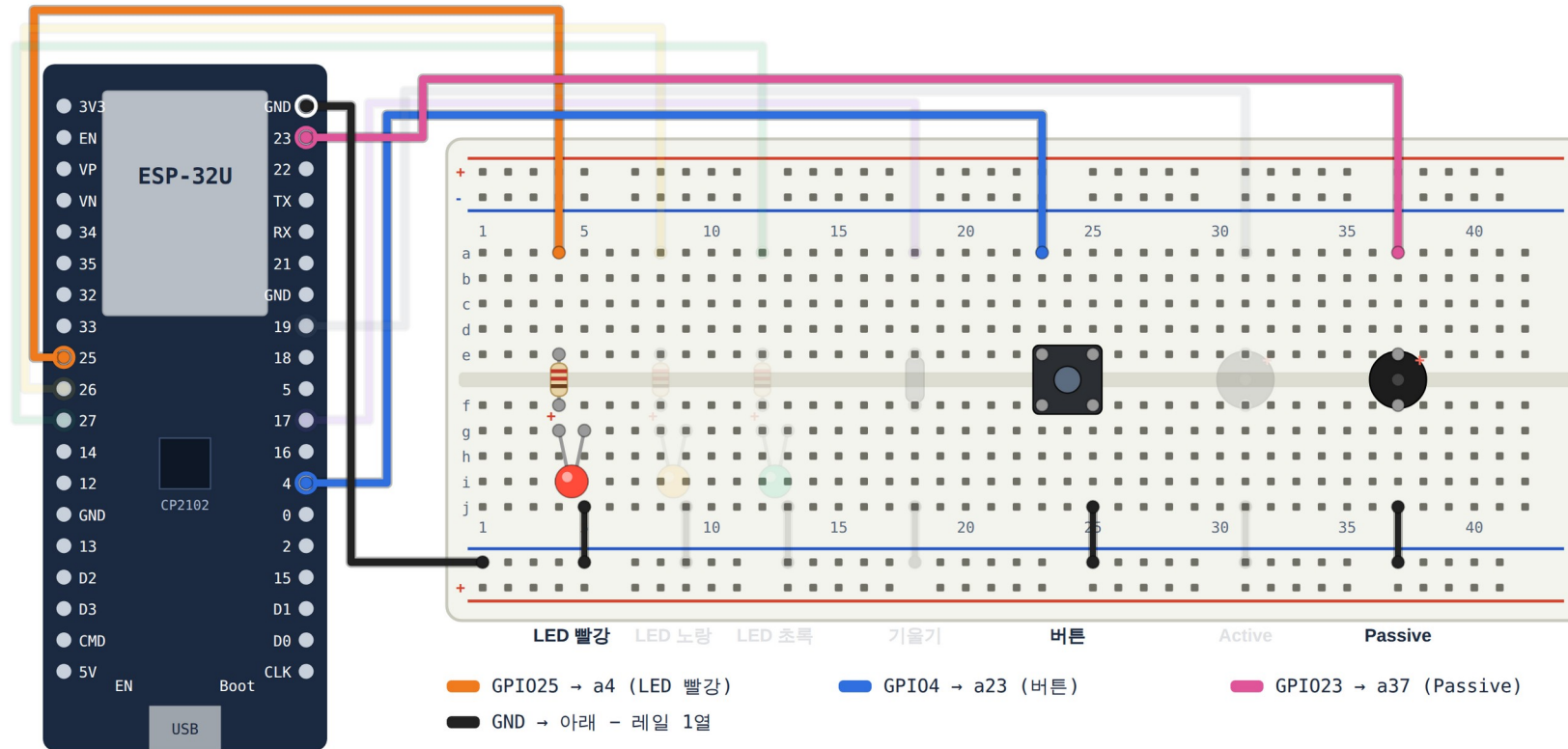
8 교시 배선 — 반응속도 게임

8 교시

그림 1 에서 LED 25 · 버튼 4 · Passive 부저 23 만 씩니다

종합 과제 8-A 배선 — LED 25 · 버튼 4 · Passive 부저 23

진한 부분만 이 예제에서 씁니다. 흐린 부분은 그림 1 대로 꽂혀 있어도 상관없습니다



7 교시에 RGB 로 옮긴 25 · 26 · 27 선은 원래 자리 (a4 · a8 · a12) 로 되돌립니다

종합 과제 8-A · 반응속도 게임

8 교시

배선 : LED 25, 버튼 4, Passive Buzzer 23 · 뼈대의 TODO 1~6 을 채웁니다

상태	할 일
IDLE	누르면 1~4 초 랜덤 대기 시간 정하고 WAIT
WAIT	TODO 1 누르면 FOUL · TODO 2 시간 지나면 LED ON, GO
GO	TODO 3 누르면 반응 시간 계산 , RESULT · TODO 4 3 초 초과면 IDLE
RESULT	TODO 5 신기록 갱신 · TODO 6 300ms 미만 GOOD, 아니면 BAD
FOUL	반칙 메시지 , BAD 멜로디 , LED 3 번 깜빡임

제출 체크리스트

- 반응 시간이 ms 단위로 출력된다
- WAIT 중 누르면 반칙 처리 , 게임이 멈추지 않는다
- 한 번 누름 = 한 번 인식 (디바운스)
- 상태가 함수 + 함수 포인터 표로 구성

반응속도 게임 — 버튼을 눌러 시작
준비 ... 불이 켜지면 누르세요
반응 시간 : 287 ms
신기록 !
버튼을 눌러 다시 시작
준비 ... 불이 켜지면 누르세요
반칙 ! 불이 켜지기 전에 눌렀습니다

확장 : 3 판 평균 (ST_SUMMARY) · 2 인 대결 (button_t
배열) · 대기 중 노랑 LED 숨쉬기 (delay 없이)

오늘의 여섯 문장

- 1 ESP32 는 3.3V 칩 . 입력 전용 (34~39) · Strapping(0·2·5·12·15) · Flash(6~11) 를 피한다
- 2 .ino → .cpp → .elf → .bin → Flash 0x10000. setup/loop 는 loopTask 안의 함수
- 3 LED 에는 항상 220Ω. 상태는 enum 으로 이름 붙이고 , 유지할 지역 변수는 static
- 4 GPIO 는 레지스터 . 한 핀만 바꿀 때는 W1TS / W1TC 에 1UL << pin
- 5 입력은 INPUT_PULLUP + 논블로킹 디바운스 . 상태는 구조체에 담아 포인터로
- 6 ISR 은 IRAM_ATTR, 짧게 , 표시만 . 공유 변수는 volatile, 읽고 - 바꾸기는 임계 구역

Day 2 예고 아날로그 입력 (ADC) — 가변저항 , 조도 , 조이스틱 , 온습도 , 가스 , 초음파 . 오늘의 button_t 처럼 센서도 구조체로 설계하고 , 링 버퍼로 이동평균 필터를 만듭니다 .